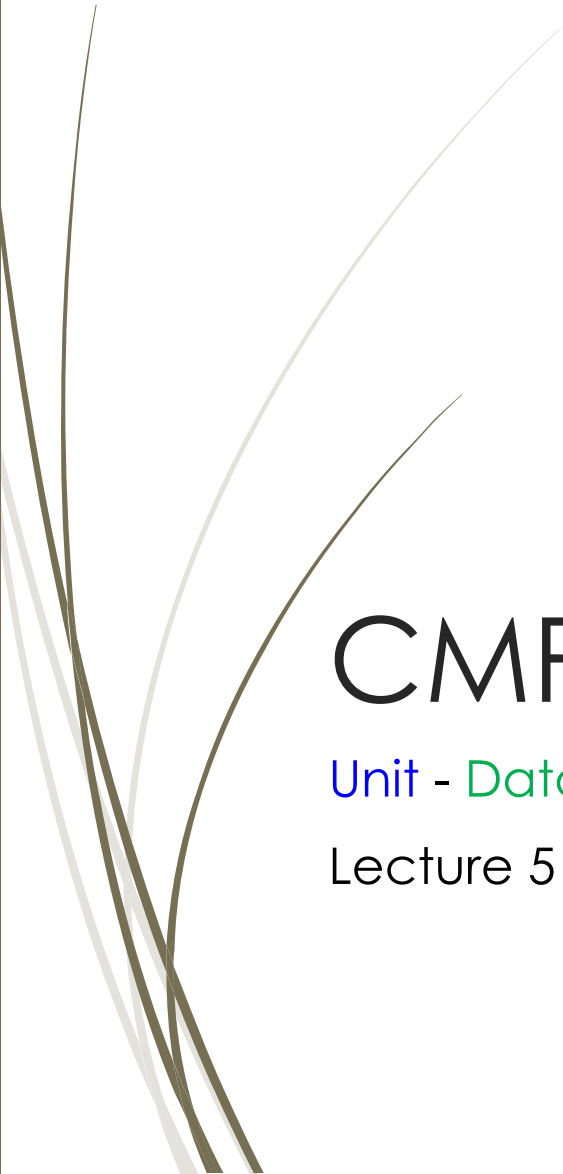




CMPT 295

Unit - Data Representation

Lecture 5 – Representing integral numbers in memory – Integer Arithmetic – Demo

Representing real numbers in memory – IEEE 754 floating point representation

Last Lecture

Addition:

Unsigned/signed:

- Behave the same way at the bit level
- Interpretation of resulting bit pattern (sum) may differ

- Unsigned addition $\rightarrow$ true sum may overflow its w bits in memory
 - If so, then actual sum $= (x + y) \bmod 2^w$ (equivalent to subtracting 2^w from true sum $(x + y)$)

- Signed addition $\rightarrow$ true sum may overflow its w bits in memory

If so then ...

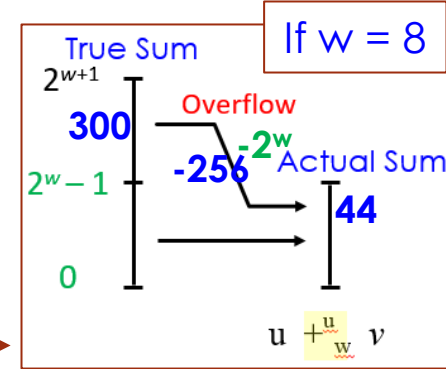
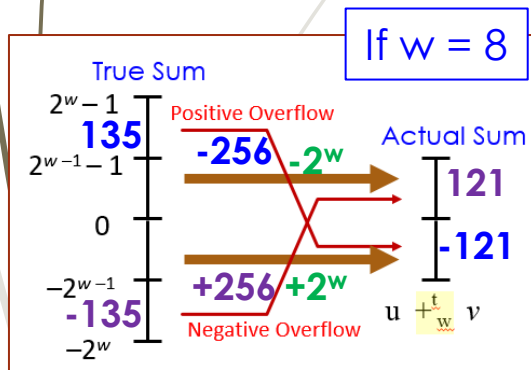
- actual sum $= \text{U2T}_w [(x + y) \bmod 2^w]$
 - $90 + 45 = 135 \bmod 256 \Rightarrow 135 - 256 = -121$
 - $-90 + -45 = -135 \bmod 256 \Rightarrow -135 + 256 = 121$
- true sum may be too +ve $\rightarrow$ positive overflow OR too -ve $\rightarrow$ negative overflow
 - $90 + 45 = 135$ but get -121
 - $-90 + -45 = -135$ but get 121

Subtraction

- Becomes an addition where the 2nd operand is transformed into its additive inverse in two's complement

- Multiplication: true product may overflow its w bits in memory, if so then...

- Unsigned: actual product $= (x * y) \bmod 2^w$
- Signed: actual product $= \text{U2T}_w [(x * y) \bmod 2^w]$
- Can be replaced by additions and shifts



Today's Menu

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

Demo

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...

Demo – Looking at integer addition in C

- What does the demo illustrate?
- Impact of fixed size memory on integer addition in C
 - Unsigned addition
 - Without overflow: when `true sum == actual sum`
 - With overflow: when `true sum != actual sum`
 - Signed addition
 - Without overflow: when `true sum == actual sum`
 - With positive overflow and negative overflow: when `true sum != actual sum`
- This demo (code and results) posted on our course web site

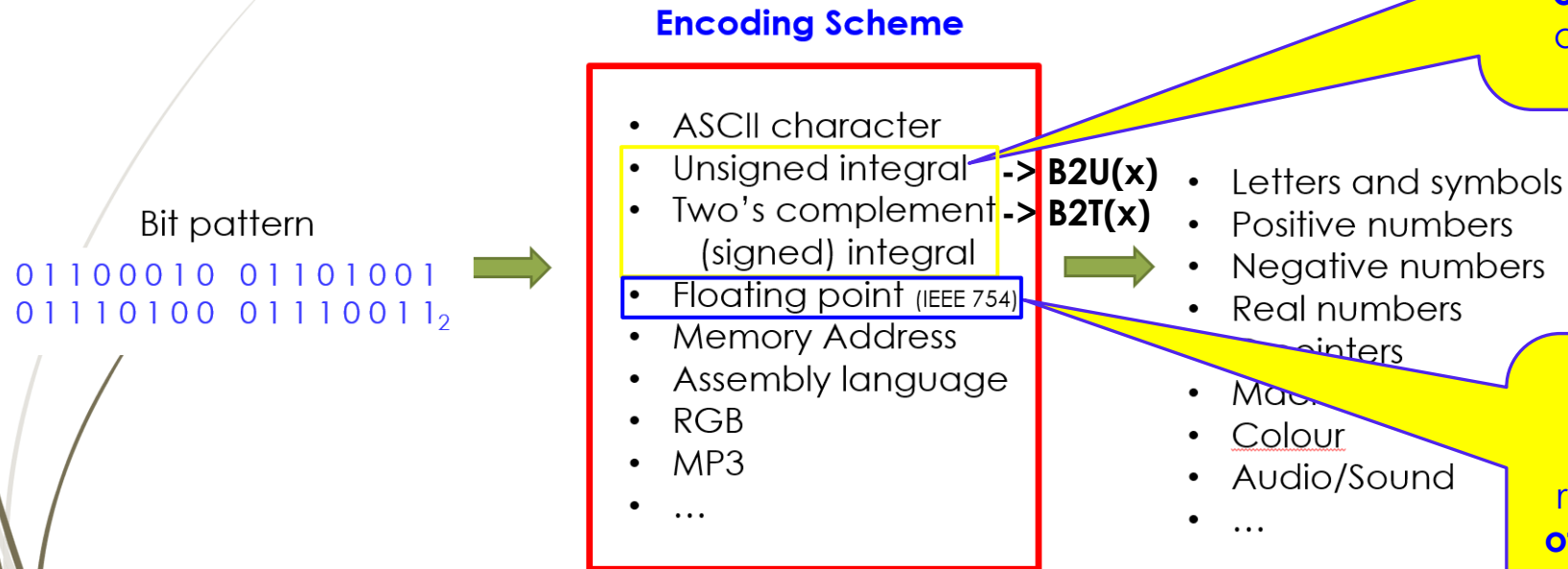
When **true sum** overflows the range of values for the data type chosen to model the variable **sum**

Lab 3 - Questions for you to think about!

- In our **C** program, how can we predict that ...
 - `true sum != actual sum`?
 - `true product != actual product`?
- In our **C** program, what can we do to mitigate the effect of arithmetic and multiplication overflow, i.e., ...
 - When `true sum != actual sum`?
 - When `true product != actual product`?

From our Lecture 3

What kind of information do **bit patterns** represent?



Integral encoding schemes (B2U(x), U2B(x), B2T(x), T2B(x)) allow us to represent a **small number of** integral values, but allows us to represent them **exactly!**

Floating point encoding scheme (B2R(x) and R2B(x)) allow us to represent a **large number of** real values, but allows us to represent them **approximately!**

Definition: An **encoding scheme** is an interpretation of a bit pattern

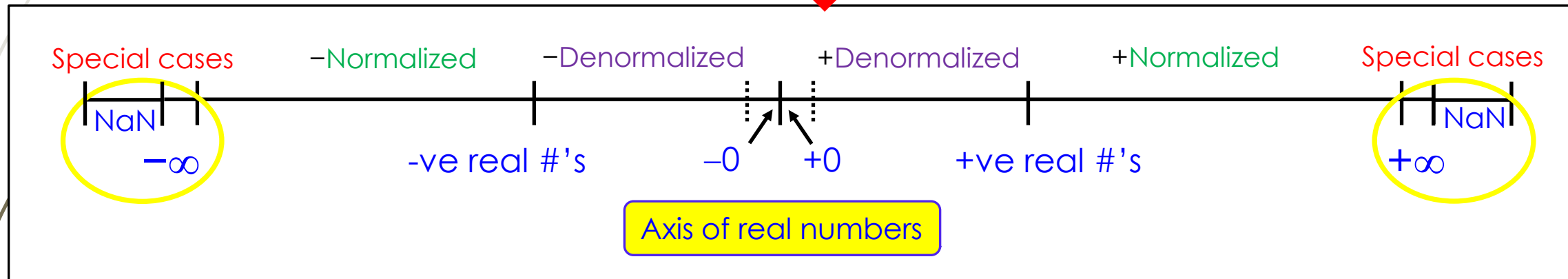
Bottom line: Which encoding scheme is used to interpret a bit pattern depends on the application currently executing (the **context**)

encoding Representing real numbers in memory

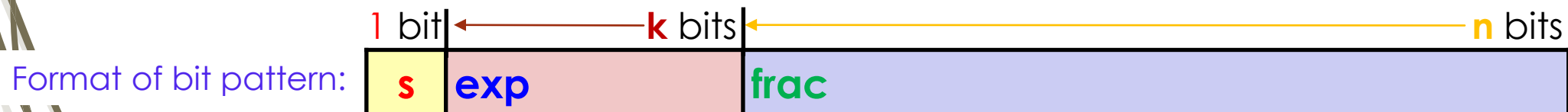
B2R(x) and R2B(x)

- Bit pattern $\Leftrightarrow$ Floating point (IEEE 754) $\Leftrightarrow$ Real numbers
or bit vector
 - Encoding scheme called IEEE 754 standard for floating point representation
- Allows us to represent a **large range** of real numbers (but not all):

And most of these real numbers are approximated.



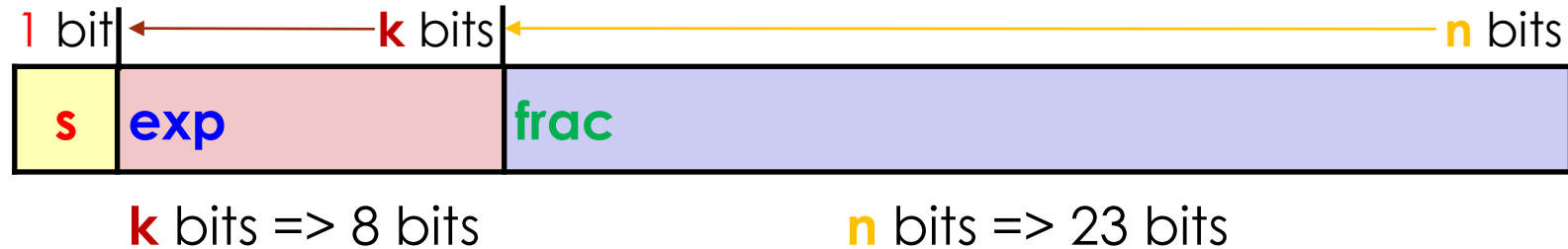
- And all represented real numbers (Normalized, Denormalized or Special cases), once converted to a bit pattern, will have this format in memory:



IEEE 754 standard - Precision options

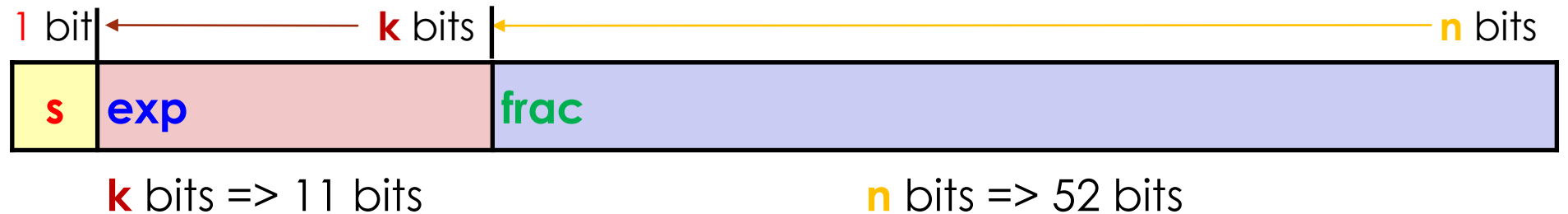
➤ **Single** precision: **32 bits** $\approx$ 7 decimal digits, range: $10^{\pm 38}$

in C:

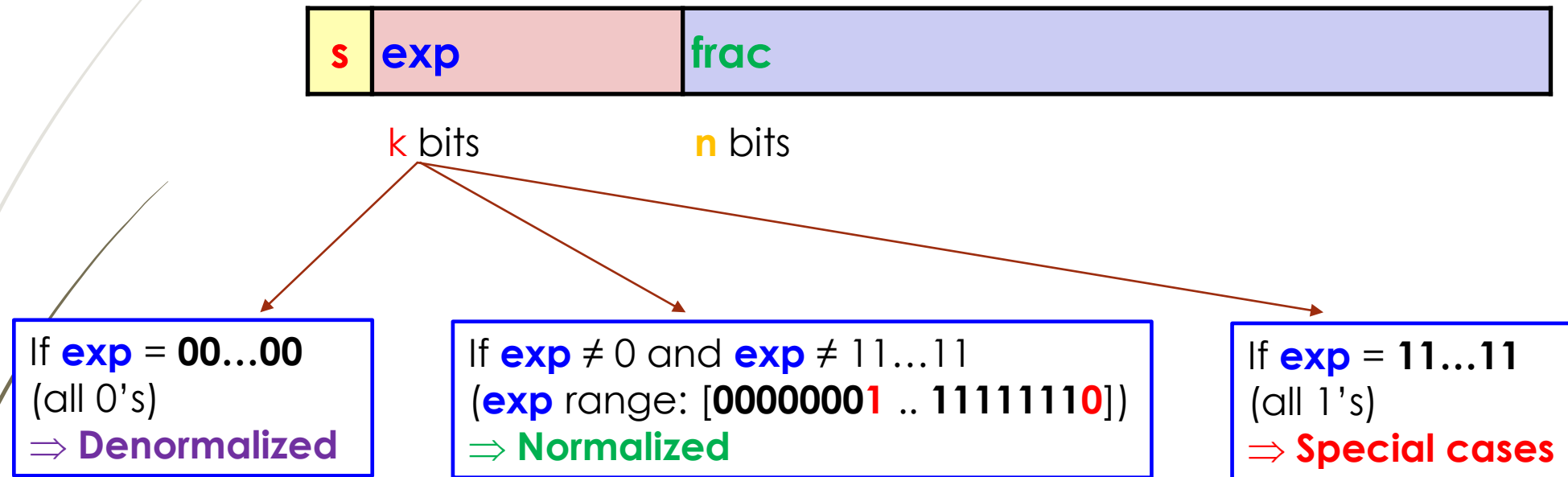


➤ **Double** precision: **64 bits** $\approx$ 16 decimal digits, range: $10^{\pm 308}$

in C:



Differentiating between Normalized, Denormalized or Special cases



Overview of IEEE 754 standard:

R2B (X) and B2R (X)

➡ **R2B (X)** - How to convert a real number (**R**) to an **IEEE 754 floating point number** (i.e., bit pattern **B** ):

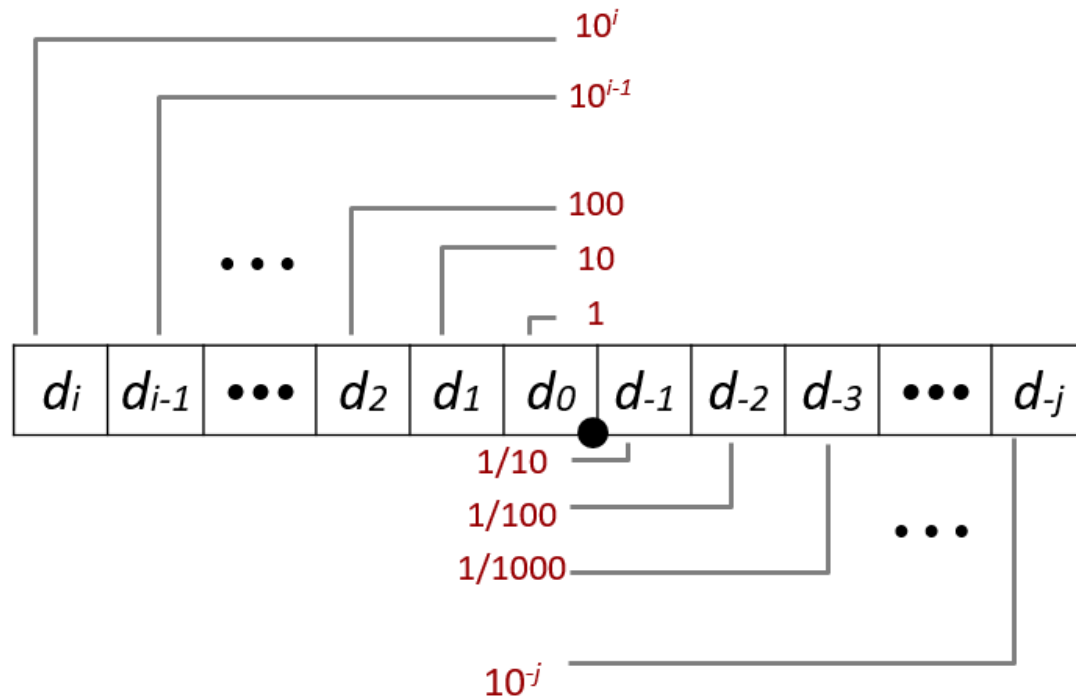
1. Convert the given real number **R** into a binary number (with integral and fractional parts as well as a binary point) using the **positional notation**.
2. Normalize this binary number into the **binary numerical form** $V = (-1)^s M 2^E$
3. Convert this binary number expressed in the **binary numerical form** **V** into an **IEEE 754 floating point number** (i.e., a bit pattern **B**) using the **appropriate** equations.

➡ **B2R (X)** - How to convert an **IEEE 754 floating point number** (**B**) to a real number (**R**):

1. Reverse the above algorithm.

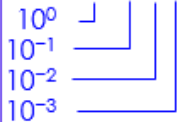
Review - Positional notation

For decimal numbers

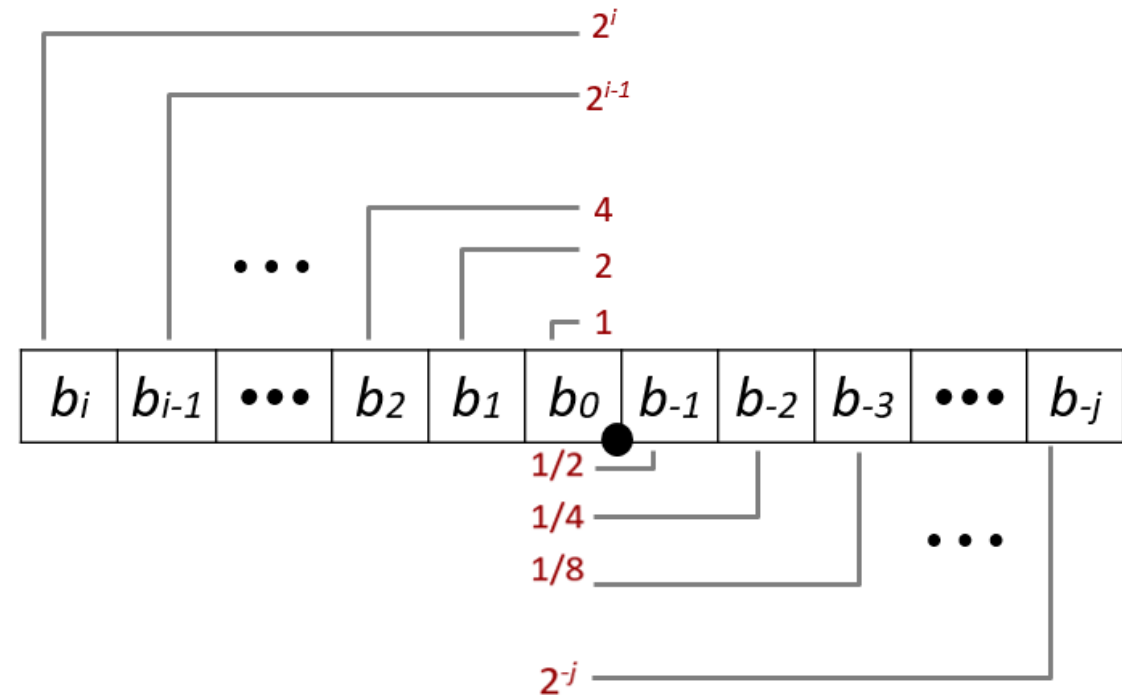


Example:

$$2.345 = 2 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2} + 5 \times 10^{-3}$$



For binary numbers



Step 1 of **R2B (X)** – Quick review of positional notation

- Convert 346.375 (= 346 3/8) to a binary number using **positional notation**
- Using the subtraction method we have already seen:

$$\begin{array}{lcl} 346.375 \rightarrow 346 - 256 = 90 \rightarrow 1 \times 2^8 & \text{MSb} & .375 - 0.5 = \text{☹} \rightarrow 0 \times 2^{-1} \\ 90 - 128 \rightarrow \text{☹} \rightarrow 0 \times 2^7 & & .375 - 0.25 = 0.125 \rightarrow 1 \times 2^{-2} \\ 90 - 64 = 26 \rightarrow 1 \times 2^6 & & .125 - 0.125 = 0 \rightarrow 1 \times 2^{-3} \\ 26 - 32 \rightarrow \text{☹} \rightarrow 0 \times 2^5 & & \\ 26 - 16 = 10 \rightarrow 1 \times 2^4 & & \\ 10 - 8 = 2 \rightarrow 1 \times 2^3 & & \\ 2 - 4 \rightarrow \text{☹} \rightarrow 0 \times 2^2 & & \\ 2 - 2 = 0 \rightarrow 1 \times 2^1 & & \\ 0 - 1 \rightarrow \text{☹} \rightarrow 0 \times 2^0 & \text{LSb} & \end{array}$$

Negative Powers
of 2

$$\begin{array}{l} 2^{-1} = 0.5 \\ 2^{-2} = 0.25 \\ 2^{-3} = 0.125 \\ 2^{-4} = 0.0625 \\ 2^{-5} = 0.03125 \\ 2^{-6} = 0.015625 \\ 2^{-7} = 0.0078125 \\ 2^{-8} = 0.00390625 \end{array}$$

Binary representation is: $1 \overset{\text{MSb}}{0} 1 \overset{\text{LSb}}{0} 1 \overset{\text{MSb}}{1} 0 \overset{\text{LSb}}{1} 0 \overset{\text{LSb}}{.} 0 \overset{\text{MSb}}{1} \overset{\text{LSb}}{1}_2$

B2R (X) – Reversing the conversion

- Convert 1011.101_2 to a real number using the **positional notation**:

$$1011.101_2 \Rightarrow$$

$$1011_2 \rightarrow 1 \times 2^3 + 1 \times 2^1 + 1 \times 2^0 = 11_{10}$$

$$.101_2 \rightarrow 1 \times 2^{-1} + 1 \times 2^{-3} = 0.5 + 0.125 = 0.625_{10}$$

$$\text{Result: } 11_{10} + 0.625_{10} = 11.625 \text{ (= } 11 \frac{5}{8}\text{)}$$

Negative Powers
of 2

$$2^{-1} = 0.5$$

$$2^{-2} = 0.25$$

$$2^{-3} = 0.125$$

$$2^{-4} = 0.0625$$

$$2^{-5} = 0.03125$$

$$2^{-6} = 0.015625$$

$$2^{-7} = 0.0078125$$

$$2^{-8} = 0.00390625$$

Summary

- Demo – Impact of fixed size memory on integer additions in **C**
- Encoding schemes **B2U(x)**, **U2B(x)**, **B2T(x)**, **T2B(x)** represent integral numbers we store in memory
 - Can encode a **small** range of integral values **exactly** (in 1, 2, 4, 8 bytes)
 - Example: We can represent the small range of integral values [-128 .. 127] **exactly** in 1 byte using a **signed char** in **C**
- Encoding scheme **IEEE 754 standard for floating point representation** represents real numbers we store in memory
 - Can encode a **much larger** range of real values, but **approximately** (in 4 or 8 bytes)
 - Example: We can represent the large range of real values $[-1.18 \times 10^{-38} \dots +3.4 \times 10^{38}]$ **approximately** in 4 bytes (single precision) using a **float** in **C**
- Overview of **IEEE 754 standard - R2B (X)** and **B2R (X)**

Next lecture

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...